1. Java Collections Framework

Java Collections Framework (JCF)

- Libraries providing:
  - Data Structures
  - Algorithms

Why JCF?

- Ready-made implementations
- Faster development
- Fewer bugs
- Optimized performance

Examples

- Sorting, Hashing, Searching

Two Parts of JCF

Part	Purpose
Data Structures	Array, List, Stack, Queue, Set, Map
Algorithms (`Collections`)	sort, reverse, binarySearch, fill

Important Implementations

Collection	Internal DS	Important Points
ArrayList	Dynamic Array	Auto-resizable
LinkedList	Doubly Linked List	Can act as Queue/Deque
Vector	Dynamic Array	Legacy, Thread-safe
Stack	Extends Vector	LIFO, Legacy

Recommendation

- Prefer `ArrayList` over `Vector`
- Prefer `ArrayDeque` over `Stack`

Set Implementations

Collection	Internal DS	Features
HashSet	Hashing	Fast, unordered
TreeSet	Red-Black Tree	Sorted, O(log n)
LinkedHashSet	Hash + Linked List	Insertion order maintained

Queue / Deque

Collection	Internal DS	Features
LinkedList	Doubly Linked List	FIFO Queue
ArrayDeque	Dynamic Array	Faster Stack/Queue
PriorityQueue	Heap	Priority-based processing

Deque

- Double Ended Queue
- Insert/delete from both ends

Implementations

- LinkedList
- ArrayDeque

Map Implementations

Collection	Internal DS	Features
HashMap	Hashing	Fast, unordered
TreeMap	Red-Black Tree	Sorted keys
LinkedHashMap	Hash + Linked List	Insertion order maintained

Set vs Map

Set	Map
Only keys	Key + Value

Collections Class

Algorithms utility class:

- sort()
- reverse()
- binarySearch()
- fill()
- max()/min()

Important Quick Points

Topic	Point
Hashing	Avg O(1)
TreeMap/TreeSet	O(log n)
HashSet	No order
LinkedHashSet/LinkedHashMap	Insertion order
TreeSet/TreeMap	Sorted order
Stack/Vector	Legacy
Map	Not part of Collection interface